

# ĐỒ ÁN: SPECRESEARCH LOOP

## Hệ thống hoàn thiện ý tưởng nghiên cứu bằng bằng chứng và vòng lặp xác nhận

Sinh viên xây dựng một website giúp người dùng chuyển một ý tưởng nghiên cứu còn mơ hồ thành một research specification rõ ràng và có kiểm chứng. Hệ thống diễn giải lại ý tưởng, phân rã problem-gap-contribution-claim-evidence, nghiên cứu các công trình liên quan, đề xuất các lựa chọn có giải thích và ví dụ để người dùng xác nhận, sau đó xây dựng kế hoạch thí nghiệm phù hợp với tài nguyên. Bản spec được đánh giá lại bởi nhiều Judge độc lập nhằm phát hiện gap yếu, claim quá rộng, citation không phù hợp hoặc thí nghiệm chưa đủ. Người dùng quyết định sửa đổi trước khi xác nhận phiên bản cuối. Sinh viên được khuyến khích sáng tạo ở từng bước nhưng phải chứng minh hiệu quả bằng dữ liệu, verifier hoặc so sánh với baseline.

### 1. Ý nghĩa của đồ án

Một ý tưởng nghiên cứu ban đầu thường còn nhiều điểm chưa rõ:

- Vấn đề nghiên cứu cụ thể là gì?
- Các nghiên cứu trước đã giải quyết đến đâu?
- Research gap có thật sự tồn tại không?
- Contribution mới nằm ở đâu?
- Claim nào cần được chứng minh?
- Thí nghiệm nào đủ để kiểm chứng claim?
- Kế hoạch có khả thi với tài nguyên hiện có không?

Nếu đặc tả ban đầu không rõ, AI hoặc nhóm nghiên cứu có thể triển khai sai hướng, đưa ra contribution quá rộng hoặc thiết kế thí nghiệm không đủ sức thuyết phục.

Sinh viên xây dựng một website giúp người dùng chuyển:

Ý tưởng còn mơ hồ

- Research spec rõ ràng
- Có nguồn minh chứng
- Có kế hoạch thí nghiệm
- Có phản biện độc lập
- Được người dùng xác nhận

Hệ thống không bảo đảm paper được chấp nhận tại AAAI, NeurIPS hoặc ICML. Hệ thống chỉ giúp đánh giá mức độ sẵn sàng của ý tưởng theo các tiêu chí như originality, significance, soundness, clarity và reproducibility.

---

Một số hình ảnh để giúp dễ hiểu hơn, sinh viên không cần làm theo y chang như vậy

[Trang chủ](#) [Dự án](#) [Lịch sử phiên bản](#) [Trợ giúp](#)

## 1. Nhập ý tưởng & Làm rõ ban đầu

Biến ý tưởng nghiên cứu mơ hồ thành mô tả rõ ràng hơn bằng câu hỏi có giải thích và ví dụ.

Ý tưởng ban đầu

“Tôi muốn xây dựng phương pháp tự động tối ưu prompt nhiều vòng để giảm hallucination khi LLM trích xuất thông tin từ paper.”

Nghiên cứu AI

Prompt Optimization

Hallucination

Paper Extraction

Phân tích ý tưởng

Cách hệ thống đang hiểu ý tưởng

Hệ thống hiểu rằng bạn muốn tạo một vòng lặp tối ưu prompt: sinh nhiều prompt, chạy thử trên cùng tập paper, phát hiện lỗi và tiếp tục cải tiến để giảm thông tin không có bằng chứng trong tài liệu nguồn.

Vấn đề chính

- Prompt thủ công có thể không ổn định
- LLM dễ tạo unsupported claims
- Cần cách kiểm tra claim-evidence rõ ràng

Mức chắc chắn: Trung bình

Câu hỏi cần xác nhận

1

Tác vụ chính là gì?

A. Trích xuất thông tin

B. Trả lời câu hỏi có dẫn nguồn

C. Tóm tắt tài liệu

D. Other

Ví dụ: lấy title, method, dataset, kết quả từ paper.

2

Bạn muốn spec cuối dùng để làm gì?

A. Làm prototype

B. Triển khai thật

C. Formal review

D. Other

Ví dụ: đủ rõ để giao cho AI hoặc nhóm dev triển khai.

3

Khi thiếu thông tin, hệ thống nên?

A. Dừng và hỏi ngay

B. Đưa ra lựa chọn để bạn chọn

C. Tạo giả định tạm thời có cảnh báo

D. Other

Ví dụ: hệ thống đề xuất 3 mức quy mô dữ liệu thay vì tự đoán.

Tóm tắt sau vòng 1

Ý tưởng

Làm rõ

3

Xác nhận

4

Sang bước tiếp theo

Gợi ý: Bạn có thể sửa trực tiếp nội dung hệ thống hiểu nếu thấy chưa đúng.

[Trang chủ](#) [Dự án](#) [Lịch sử phiên bản](#) [Trợ giúp](#)

## 2. Nghiên cứu liên quan & tìm Research Gap

Đổi sánh các công trình liên quan, rút ra khoảng trống nghiên cứu và các hướng khả thi.

Từ khóa & kế hoạch tìm kiếm

Nhập từ khóa hoặc truy vấn tìm kiếm...

prompt optimization

hallucination

claim evidence

paper extraction

Nguồn ưu tiên

Paper peer-reviewed

Proceedings chính thức

Tài liệu tác giả

Survey có nguồn rõ ràng

Bảng đối sánh related work

| Nghiên cứu           | Đã làm gì?                                                            | Loại feedback        | Điểm còn thiếu                                                                       | Nguồn |
|----------------------|-----------------------------------------------------------------------|----------------------|--------------------------------------------------------------------------------------|-------|
| OPRO (2023)          | Tối ưu prompt bằng search + score tự động để tăng chất lượng trả lời. | Score tổng tự động   | Không tách claim; không kiểm tra evidence độc lập; chưa đánh giá unsupported claims. |       |
| PromptBreeder (2023) | Tiến hóa prompt với LLM để tìm prompt tốt hơn.                        | Score tổng tự động   | Vẫn dựa trên điểm tổng; chưa dùng tin hiệu ở mức claim.                              |       |
| TextGrad (2024)      | Tối ưu prompt bằng gradient (textual) từ LLM.                         | Textual feedback     | Textual feedback khó đo lường chính xác; chưa đánh giá ở mức claim.                  |       |
| DSPy (2024)          | Framework tối ưu & biến dịch prompt cho các tác vụ.                   | Score/metric tự động | Chưa có bước xác minh evidence; chưa thủ kết hợp verifier & human-in-the-loop.       |       |

Research gap đề xuất

Các phương pháp hiện tại chủ yếu tối ưu bằng điểm tổng hoặc textual feedback. Chưa rõ việc tách output thành từng claim, kiểm tra evidence độc lập và dùng lỗi claim-level làm feedback có giúp giảm unsupported claims trong cùng ngân sách inference hay không.

Bạn muốn tập trung vào hướng nào?

A. Thuật toán tối ưu prompt

B. Claim-evidence verifier

C. Human-in-the-loop

D. Kết hợp

E. Other

Ví dụ: vừa cải tiến prompt, vừa thêm verifier và bước xác nhận người dùng.

Tóm tắt sau vòng 2

1

Nhập ý tưởng

2

Nghiên cứu liên quan & tìm Research Gap

3

Xác nhận

4

Sang bước tiếp theo

Gợi ý: Một research gap tốt phải chỉ ra nghiên cứu trước đã làm được gì, còn thiếu gì và vì sao phần thiếu đó quan trọng.

SpecResearch Loop

Trang chủ

Dự án

Lịch sử phiên bản

Trợ giúp

3. Xây dựng Contribution & Kế hoạch thí nghiệm

Biến research gap thành contribution, claim rõ ràng và kế hoạch kiểm chứng từng bước.

Contribution đề xuất

1 Framework tối ưu prompt nhiều vòng

2 Verifier claim-evidence

3 So sánh scalar vs textual vs claim-level feedback

4 Cấu hình khả thi với ngân sách giới hạn

Claim – Evidence Card

|                  |                                                                            |
|------------------|----------------------------------------------------------------------------|
| Claim            | Giảm tỉ lệ claim không có bằng chứng so với baseline.                      |
| Baseline         | Zero-shot CoT (GPT-4o)                                                     |
| Metric           | Tỉ lệ unsupported claims (%)                                               |
| Evidence         | Tỉ lệ unsupported giảm $\geq 20\%$ trên val set (300 mẫu) với $p < 0.05$ . |
| Điều kiện bác bỏ | Không đạt cải thiện $\geq 10\%$ hoặc không có ý nghĩa thống kê.            |

Kế hoạch thí nghiệm

TN1 So sánh baseline

- So sánh hệ thống đề xuất với 3 baseline.
- Đo tỉ lệ unsupported claims & factual accuracy.
- Kiểm định ý nghĩa thống kê.

TN2 Đánh giá chất lượng

- Đánh giá trên dev và val set.
- Tính đủ 2 metric: unsupported claims & factual accuracy.
- Phân tích theo từng chủ đề/dataset.

TN3 Ablation study

- Loại bỏ verifier (-V) để đo tác động.
- So sánh scalar vs textual vs claim-level feedback.
- Đo hiệu quả theo số vòng (3, 5, 10, 15).

TN4 Generalization

- Thử trên 2-3 domain/dataset ngoài benchmark.
- Kiểm tra zero-shot và few-shot.
- Đánh giá khả năng chuyển miền.

TN5 Efficiency study

- Đo thời gian, token và VRAM theo số vòng & candidate.
- So sánh cấu hình 4-bit vs 8-bit.
- Tối ưu cấu hình cho ngân sách giới hạn.

Kiểm tra tính khả thi (RTX 3090)

|              |              |                     |         |
|--------------|--------------|---------------------|---------|
| Model        | Seed prompts | Candidates mỗi vòng | Số vòng |
| 7B-8B, 4-bit | 5            | 10                  | 10      |
| Dev set      | Val set      | Top-k đầy đủ        |         |
| 50 mẫu       | 300 mẫu      | 5                   |         |

VRAM

~ 20 GB

Thời gian

~ 12-18 giờ

Token

~ 3-6 triệu

Chi phí API

tùy chọn

Nếu vượt tài nguyên, hệ thống gợi ý giảm số candidate hoặc số vòng.

Tóm tắt sau vòng 3

Ý tưởng

Làm rõ

Xác nhận

Sang bước tiếp theo

Gợi ý: Mỗi contribution nên gắn với ít nhất một thí nghiệm cụ thể.

SpecResearch Loop

Trang chủ

Dự án

Lịch sử phiên bản

Trợ giúp

4. Judge độc lập & Xác nhận bản cuối

Spec tạm thời được phản biện bởi nhiều judge độc lập trước khi người dùng quyết định sửa hay chốt bản cuối.

Spec tạm thời

1 Problem Statement

Mô tả vấn đề và bối cảnh nghiên cứu.

2 Research Gap

Khoảng trống nghiên cứu cần giải quyết.

3 Contributions

Những đóng góp chính của paper.

4 Claim-Evidence Matrix

Bảng ánh xạ claim và bằng chứng.

5 Experimental Protocol

Thiết kế thí nghiệm & chi tiết thực thi.

6 Compute Budget

Ngân sách compute & tài nguyên.

Xem chi tiết Spec tạm thời

Panel Judge độc lập

Judge 1: Gap Judge

Đánh giá khoảng trống & tính cần thiết của nghiên cứu.

Judge 2: Contribution Judge

Đánh giá đóng góp mới & mức độ khác biệt của paper.

Judge 3: Experiment Judge

Đánh giá thiết kế thí nghiệm & độ đáng tin cậy.

Judge 4: Evidence Judge

Đánh giá bằng chứng, phân tích & kết quả hỗ trợ claim.

Judge 5: Conference Readiness

Đánh giá tính phù hợp & khả năng chấp nhận của conference.

Các Judge đánh giá độc lập, không xem nhận xét của nhau.

Tổng hợp issue

4 issue

CRITICAL

Thiếu hidden test

Thiếu đánh giá tổng quát trên dữ liệu unseen.

J1, J3, J4

MAJOR

Claim tổng quát quá rộng

Claim hiện tại bao phủ quá nhiều khía cạnh.

J1, J2, J5

MAJOR

Thiếu baseline TextGrad

Cần bổ sung baseline TextGrad để so sánh.

J3, J4

MINOR

Chưa ghi rõ giới hạn token budget

Chưa nêu rõ giới hạn token cho từng mô hình.

J3, J5

Lựa chọn của người dùng

A. Thu hẹp claim

B. Mở rộng thí nghiệm

C. Chuyển thành research question

D. Other

Cách hệ thống đang hiểu lựa chọn

Hệ thống hiểu rằng bạn muốn thu hẹp claim để tập trung vào phạm vi hẹp hơn trong domain paper-science, loại bỏ các khía cạnh ngoài phạm vi.

Spec cuối cùng

Claim rõ hơn

Thí nghiệm đầy đủ hơn

Judge issue đã xử lý

Xác nhận & xuất Spec cuối

Tóm tắt sau vòng 4

Ý tưởng

Làm rõ

Xác nhận

Judge độc lập

Hoàn tất

Gợi ý: Judge giúp phản biện độc lập; người dùng vẫn là người quyết định cuối cùng.

SpecResearch Loop

NT Nguyễn Tuấn

1. Nhập ý tưởng

2. Làm rõ

3. Nghiên cứu

4. Judge

5. Spec cuối

Bản đặc tả nghiên cứu cuối

1. Problem statement

2. Research question

3. Related-work matrix

4. Research gap

5. Contributions

6. Claim – evidence matrix

7. Experimental protocol

8. Compute budget

9. Risks & limitations

10. Decision log

Để tài tập trung vào tối ưu prompt bằng feedback claim-level để giảm unsupported claim khi trích xuất thông tin từ paper khoa học.

LLM tóm tắt cách làm

1. Chọn contribution chính.

2. Đối sánh với prior work để tìm khoảng trống nghiên cứu.

3. Thiết kế thí nghiệm và chọn baseline phù hợp.

4. Xác nhận với Judge và người dùng trước khi chốt spec.

Ví dụ dễ hiểu

Trước : "Tối ưu prompt tốt hơn."

Sau : "Giảm unsupported claim bằng feedback claim-level, so sánh với OPRO và self-refine dưới cùng ngân sách inference."

Xác nhận cuối cùng

Xác nhận spec

Chỉnh sửa thêm

Xuất PDF

Xuất Markdown

Spec đã sẵn sàng cho bước triển khai hoặc viết proposal.

## 2. Mục tiêu

Website phải giúp người dùng:

- Diễn giải lại ý tưởng nghiên cứu bằng ngôn ngữ dễ hiểu.
- Tách ý tưởng thành problem, research question, gap, contribution, claim và evidence.
- Nghiên cứu các công trình liên quan từ nguồn đáng tin cậy.
- Phát hiện nội dung còn thiếu, mơ hồ hoặc chưa có bằng chứng.
- Đưa ra các lựa chọn kèm giải thích và ví dụ.
- Cho phép người dùng chọn, chỉnh sửa hoặc nhập phương án Other.
- Xây dựng kế hoạch thí nghiệm từng bước.
- Kiểm tra khả năng thực hiện với tài nguyên như RTX 3090.
- Đưa bản spec qua một số Judge độc lập.
- Để người dùng quyết định sửa đổi và xác nhận bản cuối.

### 3. Ví dụ đầu vào

Người dùng nhập:

Tôi muốn xây dựng phương pháp tự động tối ưu prompt nhiều vòng để giảm hallucination khi LLM trích xuất thông tin từ paper.

Hệ thống không viết ngay toàn bộ research proposal.

Nó phải lần lượt làm rõ:

Hallucination được định nghĩa như thế nào?  
Tác vụ trích xuất những thông tin gì?  
Prompt được tối ưu bằng phương pháp nào?  
So sánh với baseline nào?  
Contribution chính là thuật toán hay verifier?  
Dữ liệu và metric nào được sử dụng?

---

### 4. Quy trình thực hiện

#### Bước 1 — Diễn giải lại ý tưởng

Hệ thống làm gì?

Viết lại ý tưởng bằng ngôn ngữ đơn giản để kiểm tra xem hệ thống có hiểu đúng người dùng không.

Ví dụ

Hệ thống sẽ tạo nhiều phiên bản prompt, chạy chúng trên cùng một tập paper, phát hiện lỗi và tiếp tục sửa prompt. Mục tiêu là giảm những thông tin do LLM tạo ra nhưng không được paper nguồn hỗ trợ.

Sau đó hỏi:

Tôi đang hiểu đúng ý tưởng của bạn không?

Người dùng có thể:

- Xác nhận.
- Chỉnh sửa.
- Yêu cầu ví dụ khác.
- Nhập cách hiểu riêng.

Khuyến khích sáng tạo

Sinh viên có thể trực quan hóa ý tưởng thành:

- Sơ đồ.
- Concept map.
- Danh sách thành phần.
- Animation mô tả luồng nghiên cứu.

---

## Bước 2 — Phân rã ý tưởng

Hệ thống tách ý tưởng thành các thẻ nhỏ.

| Thành phần        | Ví dụ                                                               |
|-------------------|---------------------------------------------------------------------|
| Problem           | Prompt thủ công có thể không ổn định                                |
| Research question | Tối ưu nhiều vòng có giảm unsupported claims không?                 |
| Gap candidate     | Các phương pháp hiện tại chưa tối ưu trực tiếp ở mức claim–evidence |
| Contribution      | Framework tối ưu prompt dựa trên evidence feedback                  |
| Claim             | Phương pháp giảm unsupported claim                                  |
| Evidence          | Kết quả thực nghiệm trên held-out data                              |
| Constraint        | Chạy được trên RTX 3090                                             |
| Open question     | Tối ưu một prompt hay cả pipeline?                                  |

Mỗi thẻ có trạng thái:

CONFIRMED  
 PROPOSED  
 MISSING  
 AMBIGUOUS  
 UNSUPPORTED  
 CONFLICT

## Khuyến khích sáng tạo

Sinh viên có thể:

- Biểu diễn các thẻ thành graph.
  - Cho phép kéo thả để liên kết claim với evidence.
  - Dùng màu để thể hiện trạng thái.
  - Tự đề xuất thêm loại thẻ phù hợp với từng domain.
-

## Bước 3 — Nghiên cứu công trình liên quan

### Hệ thống làm gì?

Tạo từ khóa tìm kiếm, thu thập các tài liệu liên quan và lập bảng đối sánh.

### Ví dụ

| Nghiên cứu    | Đã làm gì?                                  | Feedback sử dụng  | Điểm cần nghiên cứu thêm           |
|---------------|---------------------------------------------|-------------------|------------------------------------|
| OPRO          | LLM đề xuất prompt mới từ prompt và điểm cũ | Điểm tổng         | Chưa phân tích lỗi theo từng claim |
| PromptBreeder | Tiến hóa prompt và mutation prompt          | Fitness           | Có thể tốn nhiều lần gọi model     |
| TextGrad      | Dùng phản hồi dạng văn bản                  | LLM feedback      | Judge có thể bị bias               |
| DSPy          | Tối ưu nhiều module trong pipeline          | Downstream metric | Cần định nghĩa metric rõ           |

Mỗi nhận định phải liên kết với nguồn cụ thể.

### Khuyến khích sáng tạo

Sinh viên có thể phát triển:

- Citation graph.
- Timeline nghiên cứu.
- Similarity map giữa các paper.
- Cơ chế chấm độ tin cậy của nguồn.
- Cảnh báo khi nguồn quá cũ hoặc không trực tiếp hỗ trợ claim.

---

## Bước 4 — Đề xuất research gap

Research gap không được tạo theo cách:

Tôi chưa thấy paper giống hết nên đây là gap.

Hệ thống phải làm rõ:

Nghiên cứu trước đã làm được gì?

Điểm nào vẫn còn hạn chế?

Vì sao hạn chế đó quan trọng?

Có thể kiểm nghiệm bằng thí nghiệm nào?

## Ví dụ gap candidate

Các phương pháp tối ưu prompt hiện tại có thể sử dụng điểm tổng hoặc textual feedback. Chưa rõ việc tách output thành từng claim, kiểm tra evidence độc lập và dùng lỗi claim-level làm feedback có giúp giảm unsupported claims trong cùng ngân sách inference hay không.

Người dùng được chọn:

### A. Tập trung vào thuật toán tối ưu prompt

Điểm mới nằm ở mutation, selection hoặc search.

### B. Tập trung vào claim–evidence verifier

Điểm mới nằm ở cách kiểm tra hallucination.

### C. Tập trung vào human-in-the-loop

Điểm mới nằm ở cách người dùng xác nhận và điều chỉnh quá trình.

### D. Kết hợp các hướng

Chọn một contribution chính và các contribution phụ.

### E. Other

Người dùng nhập hướng riêng.

## Khuyến khích sáng tạo

Sinh viên được phép phát triển cách tìm gap mới, nhưng phải chứng minh gap bằng tài liệu, không chỉ dựa vào đánh giá chủ quan của LLM.

---

## Bước 5 — Xây dựng contribution và claim

### Ví dụ contribution rõ ràng

1. Một framework tối ưu prompt qua nhiều vòng bằng claim-level evidence feedback.
2. Một verifier phân biệt claim có evidence, thiếu evidence và mâu thuẫn với evidence.
3. Một thực nghiệm so sánh scalar feedback, textual feedback và claim-level feedback.
4. Một cấu hình có thể chạy với ngân sách GPU hoặc API giới hạn.

## Claim–Evidence Card

Claim:

Phương pháp giảm unsupported claims.

Baseline:

Human prompt, self-refine, OPRO-style optimizer.

Metric:

Unsupported claim rate.

Evidence:

Kết quả trên validation và hidden test.

Điều kiện bác bỏ:

Không cải thiện ổn định hoặc làm giảm coverage đáng kể.

## Khuyến khích sáng tạo

Sinh viên có thể đề xuất:

- Contribution về phương pháp.
- Contribution về evaluator.
- Dataset hoặc benchmark mới.
- Visualization mới.
- Insight mới từ quá trình tối ưu.

Mỗi contribution phải có cách kiểm chứng.

---

## Bước 6 — Thiết kế thí nghiệm

Hệ thống tạo kế hoạch thí nghiệm theo từng bước.

Ví dụ

*Thí nghiệm 1 — So sánh baseline*

Human-written prompt

Self-refine

Random mutation

OPRO-style optimization

Phương pháp đề xuất

Các phương pháp phải dùng cùng:

- Model.
- Dataset.
- Token budget.

- Số lần gọi LLM.
- Điều kiện chạy.

### *Thí nghiệm 2 — Đánh giá chất lượng*

Metric có thể gồm:

- Claim precision và recall.
- Evidence support rate.
- Unsupported claim rate.
- Contradiction rate.
- Token cost.
- Latency.
- JSON validity.

### *Thí nghiệm 3 — Ablation*

Lần lượt bỏ:

- Claim decomposition.
- Evidence verifier.
- Textual feedback.
- Candidate diversity.
- User confirmation.

### *Thí nghiệm 4 — Generalization*

Kiểm tra prompt cuối trên:

- Tập dữ liệu chưa dùng trong quá trình tối ưu.
- Loại paper khác.
- Domain khác.
- Model khác nếu có đủ ngân sách.

## Khuyến khích sáng tạo

Sinh viên có thể bổ sung:

- Error analysis trực quan.
- Pareto frontier giữa chất lượng và chi phí.
- Cơ chế dừng sớm.
- Behavioral clustering.
- Active selection để chọn candidate đáng thử nhất.

---

## Bước 7 — Kiểm tra tính khả thi

Hệ thống phải ước lượng:

- Model sử dụng.
- VRAM.
- Số candidate.
- Số vòng lặp.
- Số mẫu đánh giá.
- Token hoặc API cost.
- Thời gian thực nghiệm.

### Ví dụ cấu hình

Model: 7B-8B, 4-bit

Seed prompts: 5

Candidates mỗi vòng: 10

Số vòng: 10

Development set: 50 mẫu

Validation set: 300 mẫu

Top candidates đánh giá đầy đủ: 5

Hệ thống có thể đề xuất giảm quy mô nếu vượt tài nguyên RTX 3090.

### Khuyến khích sáng tạo

Sinh viên được phép xây:

- GPU estimator.
- Cost simulator.
- Cơ chế evaluation cascade.
- Dashboard theo dõi thời gian và chi phí thực tế.

---

## Bước 8 — Tạo bản research spec

Bản spec tạm thời gồm:

1. Problem statement.
2. Research questions.
3. Related-work matrix.
4. Research gap.
5. Proposed approach.
6. Expected contributions.
7. Claim–evidence matrix.
8. Experimental protocol.
9. Baselines và metrics.
10. Ablation plan.
11. Compute budget.
12. Risks và limitations.

13. Open issues.

14. Decision history.

---

## Bước 9 — Đánh giá bằng các Judge độc lập

Sau khi có spec, hệ thống sử dụng một số Judge độc lập.

### Judge 1 — Research Gap Judge

Kiểm tra gap có thật sự được tài liệu hỗ trợ không.

### Judge 2 — Contribution Judge

Kiểm tra contribution có mới, rõ và bị phóng đại hay không.

### Judge 3 — Experiment Judge

Kiểm tra thí nghiệm có đủ để chứng minh claim không.

### Judge 4 — Evidence Judge

Kiểm tra citation có thực sự hỗ trợ nội dung đi kèm không.

### Judge 5 — Conference Readiness Judge

Đánh giá originality, significance, soundness, clarity và reproducibility.

Các Judge có thể sử dụng:

- Những LLM khác nhau.
- Cùng một LLM nhưng context và prompt độc lập.
- Rule-based verifier kết hợp LLM Judge.

Các Judge phải đánh giá riêng trước khi xem nhận xét của nhau.

### Ví dụ nhận xét

Vấn đề:

Claim về khả năng tổng quát đang quá rộng.

Lý do:

Thí nghiệm chỉ sử dụng paper khoa học.

Mức độ:

MAJOR.

Đề xuất:

Thu hẹp claim hoặc bổ sung thêm domain.

---

## Bước 10 — Người dùng quyết định sửa đổi

Hệ thống tổng hợp ý kiến Judge và đưa ra lựa chọn.

Ví dụ

Claim hiện tại nói phương pháp hoạt động trên nhiều domain, nhưng thí nghiệm chỉ có một domain. Bạn muốn xử lý như thế nào?

A. Thu hẹp claim

Chỉ khẳng định kết quả trên paper khoa học.

B. Mở rộng thí nghiệm

Bổ sung thêm domain tài chính hoặc bất động sản.

C. Chuyển thành research question

Không khẳng định khả năng tổng quát trước khi có bằng chứng.

D. Other

Người dùng nhập hướng riêng.

Sau khi người dùng chọn:

Sửa spec

- Hiển thị phần thay đổi
- Chạy lại verifier liên quan
- Judge kiểm tra lại
- Người dùng xác nhận bản cuối

---

## 5. Chức năng bắt buộc

Website cần có:

1. Nhập ý tưởng nghiên cứu.
2. Diễn giải lại ý tưởng.
3. Phân rã problem, gap, claim, contribution và evidence.
4. Tìm kiếm và quản lý nguồn.
5. Tạo bảng related work.
6. Phát hiện ambiguity và conflict.
7. Tạo lựa chọn có giải thích, ví dụ và other.
8. Lưu quyết định người dùng.
9. Sinh kế hoạch thí nghiệm.

10. Ước lượng tài nguyên.
  11. Tạo research spec.
  12. Chạy nhiều Judge độc lập.
  13. Tổng hợp điểm đồng thuận và bất đồng.
  14. Cho người dùng quyết định sửa đổi.
  15. Quản lý version và hiển thị diff.
  16. Xuất bản spec cuối cùng.
- 

## 6. Sản phẩm bàn giao

Sinh viên cần nộp:

- Website chạy được.
- Source code.
- Tài liệu kiến trúc.
- Dataset hoặc tập use case thử nghiệm.
- Prompt của Generator và các Judge.
- Cơ chế kiểm tra citation hoặc evidence.
- Ít nhất hai baseline.
- Báo cáo đánh giá hệ thống.
- Video demo.
- Một research spec hoàn chỉnh do hệ thống tạo ra.

Sinh viên được đánh giá cao khi đề xuất được một cơ chế mới và chứng minh nó giúp:

- Giảm claim không có bằng chứng.
  - Phát hiện gap tốt hơn.
  - Giảm số câu hỏi không cần thiết.
  - Giảm bias của Judge.
  - Tạo experiment plan đầy đủ hơn.
  - Giảm thời gian hoặc chi phí hoàn thiện spec.
- 

## 8. Tinh thần sáng tạo

Sinh viên không bắt buộc tất cả phải xây cùng một hệ thống giống nhau.

Có thể sáng tạo ở các hướng:

- Cách phân rã spec.
- Cách tìm và đánh giá nguồn.

- Cách phát hiện research gap.
- Cách tạo lựa chọn cho người dùng.
- Cách đo disagreement giữa các Judge.
- Cách trực quan hóa claim–evidence.
- Cách tối ưu số câu hỏi cần xác nhận.
- Cách ước lượng tài nguyên thí nghiệm.
- Cách chống citation sai.
- Cách phát hiện claim bị phóng đại.

Tuy nhiên, mọi cải tiến phải trả lời được:

Cải tiến này giải quyết vấn đề gì, được kiểm nghiệm như thế nào và kết quả có tốt hơn baseline hay không?